Dhaka Urban Management System Lab Examination

Project 2 Questions and Model Solutions

Complete all five phases. Write Phases 1–4 on paper and implement Phase 5 in Java. This document provides an examination paper without marks, followed by a model solution. Alternative designs are acceptable when their analysis, diagrams, and implementation are consistent.

Part A Examination Questions

Project scenario

Design a Dhaka Urban Management System that integrates urban services, citizen communication, IoT monitoring, and decision support. The broader platform aims to manage traffic, improve waste collection, handle complaints, support emergency response, monitor environmental conditions, and generate urban reports.

For the examination, implement a complaint-to-service workflow: a citizen submits a complaint, the system creates the appropriate urban service, selects a response route using external traffic data, updates the complaint status, and notifies subscribers.

Instructions

- Complete Phases 1–4 on paper and Phase 5 as a Java console application.
- Simulate complaints, routes, traffic information, and notifications. A live database, internet service, physical sensors, and a graphical interface are not required.
- Use all five specified patterns. Clearly distinguish the broader platform from the implemented subsystem.
- Use consistent class names and state reasonable assumptions.

Phase 1 Requirement Analysis

Question 1 Analyze the proposed system

- Describe the system background and identify three problems the proposed system addresses.
- State one primary objective and four specific objectives.
- Identify at least four actors and their responsibilities. Define the boundary between the platform and external devices or services.
- Specify four functional requirements and two non-functional requirements. At least two functional requirements must concern complaint handling and service dispatch.

Phase 2 Object Identification

Question 2 Identify entity control and boundary objects

- Identify at least five entity classes, giving two or more attributes for each.
- Identify three control classes and explain their responsibilities.
- Identify three boundary classes and explain their external interactions.
- Present the classifications in tables and explain any difference between an external actor and its internal software representation.

Phase 3 CRC Card Development

Question 3 Prepare six CRC cards

Prepare six CRC cards for the complaint-handling and service-dispatch workflow. Include class name, at least two responsibilities, and collaborating classes. Across the six cards, include at least one entity, one control, and one boundary class.

Phase 4 UML Modeling

Question 4 Model the system structure and behavior

a. Draw a UML class diagram with at least eight classes or interfaces. Include important attributes, methods, visibility indicators, relevant associations and multiplicities, and interface realizations. Show the participants in the selected patterns. Use aggregation or composition only when ownership and lifecycle semantics justify it.

b. Draw a sequence diagram for dispatching a response to an OPEN complaint. Show complaint retrieval, service creation, external traffic retrieval through an adapter, route selection, the status change to IN_PROGRESS, and observer notification. Use an alt fragment for a route available versus no route available. When no route exists, the complaint must remain OPEN.

Phase 5 Java Implementation

Question 5 Implement and demonstrate five design patterns

Pattern	Required implementation
Singleton	UrbanDatabase provides controlled access to one shared in-memory complaint repository. Demonstrate instance identity.
Factory	UrbanServiceFactory creates WasteService, TrafficService, or EmergencyService through UrbanService. Reject unknown types. A Simple Factory is acceptable; name the variant.
Observer	Publish complaint status changes to CitizenNotifier and GovernmentDashboard. Demonstrate subscription, removal, and citizen-specific notification filtering.
Strategy	RoutePlanner switches between ShortestRouteStrategy and FastestRouteStrategy at runtime.
Adapter	TrafficAPIAdapter translates a simulated external traffic API into the TrafficServiceAPI interface, converting metres to kilometres and seconds to minutes.

Examination rules

Use status transitions OPEN → IN_PROGRESS → RESOLVED. Submission creates OPEN; dispatch changes OPEN to IN_PROGRESS only after a route is obtained; resolution is allowed only from IN_PROGRESS. Reject duplicate complaint IDs, missing IDs, unsupported service types, and invalid transitions.

Choose among three candidate routes to the same destination. Route A: 5 km, 30 minutes; Route B: 8 km, 20 minutes; Route C: 6.5 km, 25 minutes. Shortest chooses minimum distance; fastest chooses minimum estimated time. Break ties by route ID in ascending lexical order. Reject an empty candidate list.

Required Demonstrations and Submission

Test	Input or action	Expected behavior
1	Request repository twice	Same instance
2	Create WASTE, TRAFFIC, EMERGENCY	Correct concrete service types
3	Request UNKNOWN service	Reject request
4	Submit valid complaint	Stored with OPEN status
5	Repeat ID or use blank ID	Reject without a new complaint
6	Convert route A provider values	5000 m → 5 km; 1800 s → 30 min
7	Switch shortest to fastest	A first, then B
8	Dispatch valid OPEN complaint	Selected route; IN_PROGRESS; observers notified
9	Resolve an OPEN complaint	Reject; remains OPEN
10	Unsubscribe dashboard then resolve	RESOLVED; citizen notified, dashboard not notified
11	Dispatch RESOLVED or missing ID	Reject; no valid transition
12	No candidate routes during dispatch	Reject; complaint remains OPEN

Submit handwritten Phases 1–4, Java source files, compilation and execution instructions, and test results showing expected and actual behavior. The candidate-route values above are fictional examination data, not observations of Dhaka traffic.

Part B Model Solutions

This solution implements complaint handling, simulated service assignment, route selection, and notifications. Traffic control, waste-truck scheduling, environmental monitoring, live emergency dispatch, and analytics remain broader-platform extensions. The Java example is a teaching simulation.

Phase 1 Requirement Analysis Solution

Background and problems

A shared platform can coordinate reports from citizens and the urban teams that respond to them. For this examination, assume three problems: complaints are recorded in disconnected channels, citizens cannot follow status changes, and teams choose routes without a consistent comparison of distance and travel time. These are scenario assumptions rather than verified findings about Dhaka institutions.

Objectives

Primary objective: build an object-oriented platform that records complaints, coordinates response services, selects routes, and communicates progress.

- Record complaints with unique identifiers and service categories.
- Assign the appropriate urban service through a creation abstraction.
- Select routes dynamically using a distance or travel-time policy.
- Notify citizens and the administrator dashboard of valid status updates.

Phase 1 Actors Requirements and Scope

Actors and system boundary

Actor	Interaction
Citizen	Submits complaints and receives updates for their own complaints.
City Administrator	Monitors the dashboard and configures the active routing policy.
Waste Manager	Coordinates the waste team assigned to waste complaints.
Traffic Officer	Coordinates responses to traffic complaints.
Emergency Officer	Coordinates simulated emergency-service responses.
External Traffic Provider	Supplies candidate-route distances and estimated times.

IoT Sensor and Government Authority are broader-platform actors for monitoring and oversight. Physical sensors and third-party traffic providers are external. A Sensor class or TrafficAPIAdapter inside the platform is an internal representation or communication component, not the external actor itself.

Functional requirements

- FR1: Validate and store a complaint with a unique nonblank ID, citizen ID, supported service category, location, and OPEN status.
- FR2: Create the selected urban service, obtain candidate routes through an adapter, and dispatch only when a route is available.
- FR3: Allow runtime selection of shortest-distance or fastest-time routing and enforce the specified status transitions.
- FR4: Publish submission and status-change events to registered subscribers, filtering citizen-facing messages by citizen ID.

Non functional requirements

- Maintainability: new routing strategies and traffic-provider adapters should implement stable interfaces without changing complaint coordination.
- Reliability: invalid requests must be rejected without an invalid status change; dispatch failure before assignment must leave an OPEN complaint OPEN.

Assumptions

Use one JVM and a single-threaded console workflow with in-memory storage. Service categories are uppercase WASTE, TRAFFIC, and EMERGENCY. Candidate routes share the destination and are returned in a provider-defined format. Notifications are synchronous console messages. Citizen IDs simulate recipients; authentication and access control are not implemented.

Phase 2 Entity Object Solution

Entity	Attributes	Scope
Complaint	id, citizenId, serviceType, location, status	Implemented
Route	id, km, minutes	Implemented; supports route decisions
Citizen	citizenId, name	Extension; code uses citizenId only
WasteBin	binId, fillLevel, location	Extension
Location	locationId, name, coordinates	Extension; code uses a location string

Control and Boundary Object Solution

Control objects

Class	Responsibility
ComplaintManager	Validate submission; coordinate dispatch and resolution; enforce transitions and publish status.
RoutePlanner	Obtain adapted route candidates and delegate selection to the active strategy.
UrbanAnalyticsEngine	Generate urban service reports; broader-platform extension.

UrbanServiceFactory and StatusPublisher are additional creation and notification coordination services. They are used in the implementation even though only three control objects were requested.

Boundary objects

Class	Responsibility
CitizenMobileApp	Citizen interaction in the broader design; simulated by DUMSExam.main in the executable.
GovernmentDashboard	Receive and display complaint status notifications.
TrafficAPIAdapter	Translate provider results into the platform route interface.

CitizenNotifier is another output boundary. UrbanDatabase is the repository service, while Complaint and Route represent domain data. The external human Citizen actor and a possible Citizen data class serve different roles. Classification depends on the stated system boundary.

Phase 3 CRC Card Solution

Each card below states responsibilities and collaborators. Extension entities do not need to appear in the executable subsystem.

ComplaintManager

Responsibilities: validate and record submissions; coordinate service creation and route planning; enforce dispatch and resolution transitions.
Collaborators: UrbanDatabase, Complaint, UrbanServiceFactory, UrbanService, RoutePlanner, StatusPublisher.

Complaint

Responsibilities: retain complaint identity, owner, category, and location; expose current status; accept status updates coordinated by the manager.
Collaborators: ComplaintManager, UrbanDatabase, UrbanService, StatusPublisher.

RoutePlanner

Responsibilities: request route candidates; delegate route choice; replace the active routing policy at runtime.
Collaborators: TrafficServiceAPI, RouteStrategy, Route.

TrafficAPIAdapter

Responsibilities: request external candidate data; translate field types and units into validated Route objects.
Collaborators: ExternalTrafficAPI, TrafficServiceAPI, Route.

StatusPublisher

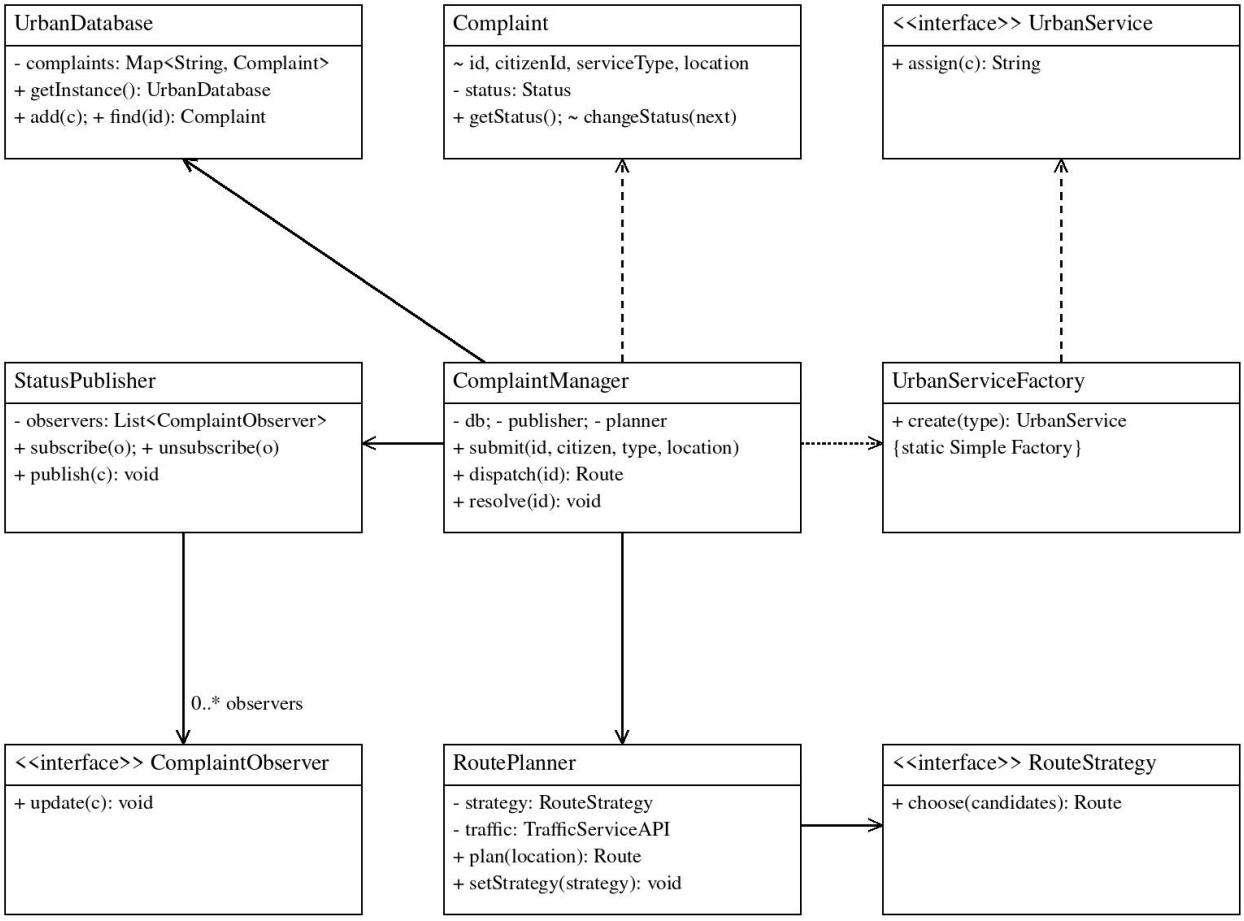
Responsibilities: maintain subscribers; remove subscribers; deliver complaint events to registered observers.
Collaborators: ComplaintObserver, ComplaintManager, Complaint.

CitizenNotifier

Responsibilities: filter events by citizen ID; display matching status updates; track delivery count for testing.
Collaborators: ComplaintObserver, StatusPublisher, Complaint.

Phase 4 UML Class Diagram Solution

The core diagram uses solid navigable associations for retained references and dashed arrows for method dependencies. Package visibility is denoted by ~. Constructors and helper details are omitted for readability. The relationship table completes the concrete pattern participants.

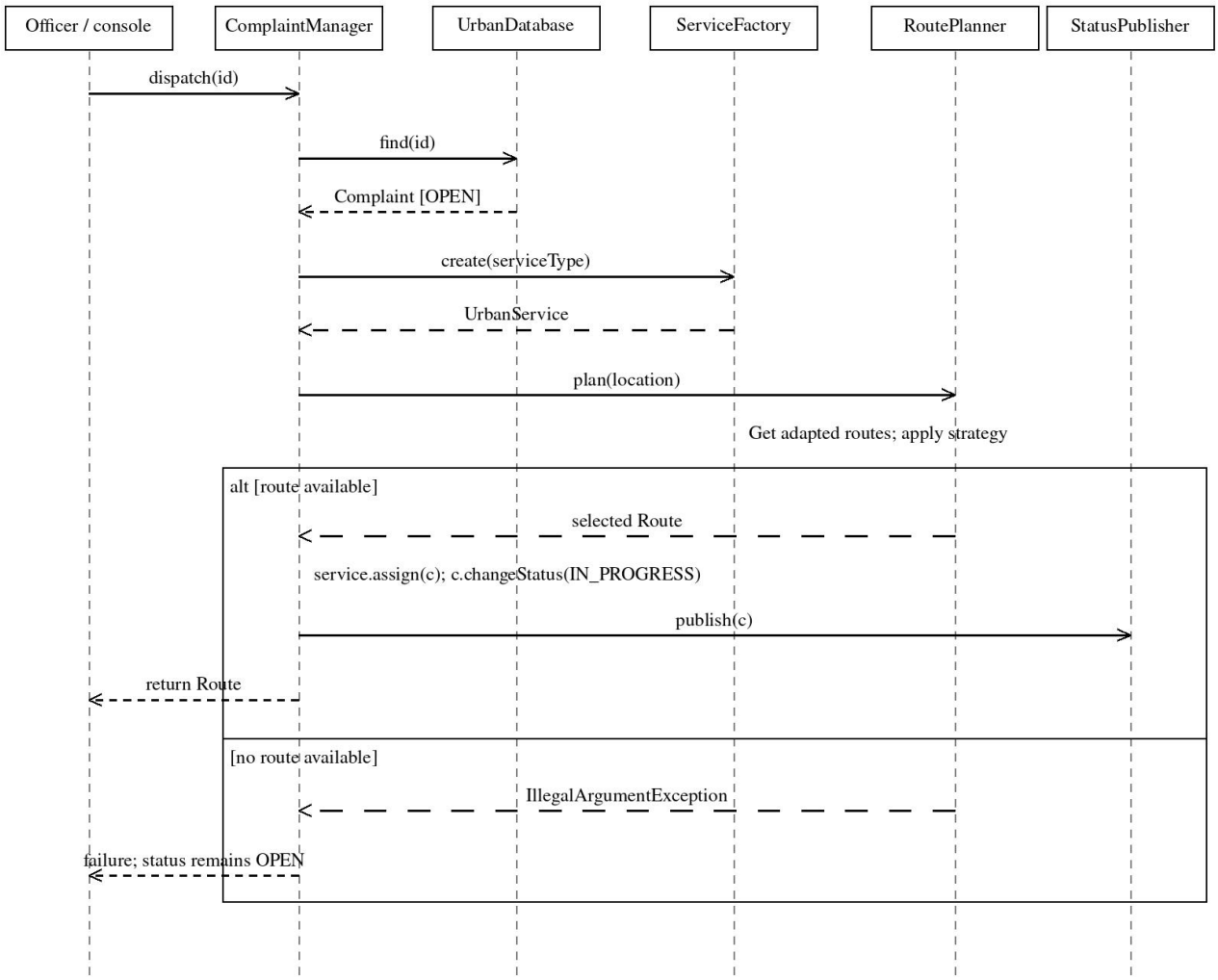


Manager holds one repository, publisher, and planner; planner holds one active strategy.

Pattern or relationship	Additional UML details
Repository association	UrbanDatabase references 0..* Complaint objects. A complaint can exist before insertion; composition is not assumed.
Factory realization	WasteService, TrafficService, and EmergencyService each realize UrbanService. UrbanServiceFactory depends on these concrete types.
Observer realization	CitizenNotifier and GovernmentDashboard realize ComplaintObserver.
Strategy realization	ShortestRouteStrategy and FastestRouteStrategy realize RouteStrategy.
Adapter realization	TrafficAPIAdapter realizes TrafficServiceAPI, holds one ExternalTrafficAPI, and creates Route objects. RoutePlanner holds one TrafficServiceAPI.

Phase 4 UML Sequence Diagram Solution

A service officer initiates dispatch through the console. A valid OPEN complaint has already been recorded. The traffic retrieval inside RoutePlanner is expanded in the explanation following the diagram.



Adapter calls: RoutePlanner.plan() → TrafficAPIAdapter.routes() → ExternalTrafficAPI.fetchCandidates(). The adapter converts metres and seconds into kilometres and minutes. The selected RouteStrategy chooses among the converted candidates. An empty list raises an exception before assignment or a status change.

Observer calls: StatusPublisher.publish(c) invokes update(c) on each registered observer. CitizenNotifier displays only events matching its citizenId; GovernmentDashboard displays every received event. This recipient filter is a simulation, not an authorization system.

A normal dispatch first obtains a route, then simulates service assignment, changes status, and publishes the update. No composition diamonds are required: ownership and object lifetimes are not strong whole–part relationships in this example.

Phase 5 Java Implementation Solution

Save the following complete listing as DUMSExam.java. It uses nested classes so the examination solution can be compiled as one file. Helper classes can instead be moved to separate files without changing the pattern roles.

Compile: javac DUMSExam.java
Run: java DUMSExam

The implementation uses an eager Singleton for the repository, a Simple Factory for service creation, Observer subscriptions, interchangeable route Strategies, and an Adapter for traffic data. The Singleton is one instance per class-loader scope, not a claim of one physical database across servers.

Route selection minimizes one metric over the provider's candidate list. It is not a shortest-path search over a road graph, nor a complete dispatch scheduler. The API returns fixed fictional candidates for any nonblank destination.

Status mutation is package-visible and coordinated by ComplaintManager. Direct domain mutation outside the manager should be avoided. The repository is in memory and is not designed for concurrent access.

Verification: the code was compiled and executed using OpenJDK 17.0.20. The test-results section reports the actual console checks. The example does not require third-party Java libraries.

Java Solution Listing 1

```
import java.util.*;

public class DUMSExam {
  enum Status { OPEN, IN_PROGRESS, RESOLVED }
  static final class Complaint {
    final String id, citizenId, serviceType, location;
    private Status status = Status.OPEN;
    Complaint(String id, String citizenId, String type, String location) {
      this.id = id; this.citizenId = citizenId;
      this.serviceType = type; this.location = location;
    }
    public Status getStatus() { return status; }
    void changeStatus(Status next) { status = next; }
  }
  static final class UrbanDatabase {
    private static final UrbanDatabase INSTANCE = new UrbanDatabase();
    private final Map<String, Complaint> complaints = new LinkedHashMap<>();
    private UrbanDatabase() { }
    public static UrbanDatabase getInstance() { return INSTANCE; }
    public void add(Complaint c) {
      if (complaints.containsKey(c.id))
        throw new IllegalArgumentException("Duplicate complaint ID");
      complaints.put(c.id, c);
    }
    public Complaint find(String id) {
      Complaint c = complaints.get(id);
      if (c == null) throw new IllegalArgumentException("Unknown complaint");
      return c;
    }
  }
  interface UrbanService { String assign(Complaint c); }
  static final class WasteService implements UrbanService {
    public String assign(Complaint c) { return "Waste team -> " + c.location; }
  }
  static final class TrafficService implements UrbanService {
    public String assign(Complaint c) { return "Traffic team -> " + c.location; }
  }
  static final class EmergencyService implements UrbanService {
    public String assign(Complaint c) { return "Emergency team -> " + c.location; }
  }
  static final class UrbanServiceFactory {
    public static UrbanService create(String type) {
      if ("WASTE".equals(type)) return new WasteService();
```

Java Solution Listing 2

```
    if ("TRAFFIC".equals(type)) return new TrafficService();
    if ("EMERGENCY".equals(type)) return new EmergencyService();
    throw new IllegalArgumentException("Unsupported service type");
  }
 }
 interface ComplaintObserver { void update(Complaint c); }
 static final class CitizenNotifier implements ComplaintObserver {
  private final String citizenId;
  int received;
  CitizenNotifier(String id) { citizenId = id; }
  public void update(Complaint c) {
   if (!citizenId.equals(c.citizenId)) return;
   received++;
   System.out.println("Citizen " + citizenId + ": " + c.id + " " + c.getStatus());
  }
 }
 static final class GovernmentDashboard implements ComplaintObserver {
  int received;
  public void update(Complaint c) {
   received++;
   System.out.println("Dashboard: " + c.id + " " + c.getStatus());
  }
 }
 static final class StatusPublisher {
  private final List<ComplaintObserver> observers = new ArrayList<>();
  public void subscribe(ComplaintObserver o) {
   if (!observers.contains(o)) observers.add(o);
  }
  public void unsubscribe(ComplaintObserver o) { observers.remove(o); }
  public void publish(Complaint c) {
   for (ComplaintObserver o : new ArrayList<>(observers)) o.update(c);
  }
 }
 static final class Route {
  final String id;
  final double km, minutes;
  Route(String id, double km, double minutes) {
   if (!Double.isFinite(km) || !Double.isFinite(minutes)
       || km <= 0 || minutes <= 0)
     throw new IllegalArgumentException("Invalid route measurement");
   this.id = id; this.km = km; this.minutes = minutes;
  }
 }
```

Java Solution Listing 3

```
 interface TrafficServiceAPI { List<Route> routes(String destination); }
 static final class ExternalTrafficAPI {
  // Incompatible provider format: ID | metres | seconds.
  public String[][] fetchCandidates(String destination) {
   if (destination == null || destination.trim().isEmpty())
    throw new IllegalArgumentException("Destination required");
   return new String[][] {
    {"A", "5000", "1800"},
    {"B", "8000", "1200"},
    {"C", "6500", "1500"}
   };
  }
 }
 static final class TrafficAPIAdapter implements TrafficServiceAPI {
  private final ExternalTrafficAPI api;
  TrafficAPIAdapter(ExternalTrafficAPI api) { this.api = api; }
  public List<Route> routes(String destination) {
   List<Route> result = new ArrayList<>();
   for (String[] row : api.fetchCandidates(destination)) {
    result.add(new Route(row[0], Double.parseDouble(row[1]) / 1000.0,
                  Double.parseDouble(row[2]) / 60.0));
   }
   return result;
  }
 }
 interface RouteStrategy { Route choose(List<Route> candidates); }
 static final class ShortestRouteStrategy implements RouteStrategy {
  public Route choose(List<Route> candidates) {
   return candidates.stream().min(Comparator
      .comparingDouble((Route r) -> r.km).thenComparing(r -> r.id))
      .orElseThrow(() -> new IllegalArgumentException("No routes"));
  }
 }
 static final class FastestRouteStrategy implements RouteStrategy {
  public Route choose(List<Route> candidates) {
   return candidates.stream().min(Comparator
      .comparingDouble((Route r) -> r.minutes).thenComparing(r -> r.id))
      .orElseThrow(() -> new IllegalArgumentException("No routes"));
  }
 }
 static final class RoutePlanner {
  private RouteStrategy strategy;
  private final TrafficServiceAPI traffic;
```

Java Solution Listing 4

```
    RoutePlanner(RouteStrategy strategy, TrafficServiceAPI traffic) {
      this.strategy = strategy; this.traffic = traffic;
    }
    public void setStrategy(RouteStrategy strategy) { this.strategy = strategy; }
    public Route plan(String location) {
      return strategy.choose(traffic.routes(location));
    }
  }
  static final class ComplaintManager {
    private final UrbanDatabase db;
    private final StatusPublisher publisher;
    private final RoutePlanner planner;
    ComplaintManager(UrbanDatabase db, StatusPublisher publisher, RoutePlanner planner) {
      this.db = db; this.publisher = publisher; this.planner = planner;
    }
    public Complaint submit(String id, String citizen, String type, String location) {
      if (id == null || id.trim().isEmpty() || citizen == null
          || citizen.trim().isEmpty() || location == null || location.trim().isEmpty())
        throw new IllegalArgumentException("ID, citizen, and location required");
      UrbanServiceFactory.create(type); // Validate before storage.
      Complaint c = new Complaint(id, citizen, type, location);
      db.add(c); publisher.publish(c);
      return c;
    }
    public Route dispatch(String id) {
      Complaint c = db.find(id);
      if (c.getStatus() != Status.OPEN)
        throw new IllegalStateException("Only OPEN complaints can be dispatched");
      UrbanService service = UrbanServiceFactory.create(c.serviceType);
      Route route = planner.plan(c.location); // May fail: keep OPEN if it does.
      String assignment = service.assign(c);
      c.changeStatus(Status.IN_PROGRESS); publisher.publish(c);
      System.out.println(assignment + "; route=" + route.id);
      return route;
    }
    public void resolve(String id) {
      Complaint c = db.find(id);
      if (c.getStatus() != Status.IN_PROGRESS)
        throw new IllegalStateException("Only IN_PROGRESS complaints can be resolved");
      c.changeStatus(Status.RESOLVED); publisher.publish(c);
    }
  }
  static void expect(boolean condition, String label) {
```

Java Solution Listing 5

```
    if (!condition) throw new AssertionError(label);
    System.out.println("PASS: " + label);
  }
  static void rejects(Runnable action, String label) {
    try { action.run(); }
    catch (IllegalArgumentException | IllegalStateException e) {
      System.out.println("PASS: " + label); return;
    }
    throw new AssertionError(label);
  }
  public static void main(String[] args) {
    UrbanDatabase db = UrbanDatabase.getInstance();
    expect(db == UrbanDatabase.getInstance(), "singleton identity");
    expect(UrbanServiceFactory.create("WASTE") instanceof WasteService, "waste factory");
    expect(UrbanServiceFactory.create("TRAFFIC") instanceof TrafficService, "traffic factory");
    expect(UrbanServiceFactory.create("EMERGENCY") instanceof EmergencyService,
           "emergency factory");
    rejects(() -> UrbanServiceFactory.create("UNKNOWN"), "unsupported service");
    TrafficServiceAPI traffic = new TrafficAPIAdapter(new ExternalTrafficAPI());
    List<Route> routes = traffic.routes("Mirpur");
    expect(routes.get(0).km == 5.0 && routes.get(0).minutes == 30.0,
           "adapter units");
    RoutePlanner planner = new RoutePlanner(new ShortestRouteStrategy(), traffic);
    expect("A".equals(planner.plan("Mirpur").id), "shortest chooses A");
    planner.setStrategy(new FastestRouteStrategy());
    expect("B".equals(planner.plan("Mirpur").id), "fastest chooses B");
    rejects(() -> new ShortestRouteStrategy().choose(Collections.emptyList()),
            "empty route list");
    StatusPublisher publisher = new StatusPublisher();
    CitizenNotifier citizen = new CitizenNotifier("U1");
    CitizenNotifier other = new CitizenNotifier("U2");
    GovernmentDashboard dashboard = new GovernmentDashboard();
    publisher.subscribe(citizen); publisher.subscribe(other); publisher.subscribe(dashboard);
    ComplaintManager manager = new ComplaintManager(db, publisher, planner);
    Complaint c = manager.submit("C1", "U1", "WASTE", "Mirpur");
    expect(c.getStatus() == Status.OPEN, "submission OPEN");
    expect(citizen.received == 1 && dashboard.received == 1 && other.received == 0,
           "subscribers and citizen filtering");
    rejects(() -> manager.submit("C1", "U1", "WASTE", "Mirpur"), "duplicate ID");
    rejects(() -> manager.submit("", "U1", "WASTE", "Mirpur"), "blank ID");
    rejects(() -> manager.resolve("C1"), "cannot resolve OPEN complaint");
    Route chosen = manager.dispatch("C1");
    expect("B".equals(chosen.id) && c.getStatus() == Status.IN_PROGRESS,
```

Java Solution Listing 6

```
            "dispatch fastest and change status");
        expect(citizen.received == 2 && dashboard.received == 2, "dispatch notifications");
        publisher.unsubscribe(dashboard);
        manager.resolve("C1");
        expect(c.getStatus() == Status.RESOLVED && citizen.received == 3
            && dashboard.received == 2, "resolve and unsubscription");
        rejects(() -> manager.dispatch("C1"), "cannot redispatch RESOLVED complaint");
        rejects(() -> manager.dispatch("missing"), "unknown complaint ID");
        RoutePlanner emptyPlanner = new RoutePlanner(new FastestRouteStrategy(),
            destination -> Collections.emptyList());
        ComplaintManager noRoute = new ComplaintManager(db, publisher, emptyPlanner);
        Complaint c2 = noRoute.submit("C2", "U1", "TRAFFIC", "Dhanmondi");
        int before = citizen.received;
        rejects(() -> noRoute.dispatch("C2"), "no-route dispatch");
        expect(c2.getStatus() == Status.OPEN && citizen.received == before,
            "no-route preserves OPEN");
        System.out.println("All checks completed.");
    }
}
```

Testing Results and Model Interpretation

The main method checks repository identity, concrete service types, adapter units, both routing policies, complaint transitions, notification filtering, and subscriber removal. expect() throws AssertionError on failure; rejects() verifies that invalid operations throw the specified application exceptions.

Test group	Observed result
Repository and factory	Identity matched; all three service types created; unknown type rejected.
Adapter and route strategies	Route A converted to 5 km and 30 minutes; shortest chose A; fastest chose B; empty route list rejected.
Submission	Complaint C1 stored OPEN; duplicate and blank IDs rejected.
Notification filtering	U1 and the dashboard received submission; U2 did not receive U1's event.
Dispatch	OPEN C1 assigned route B and changed to IN_PROGRESS; both active recipients notified.
Resolution and removal	Resolving OPEN was rejected; after dispatch and dashboard removal, resolution notified only U1.
Invalid requests	Redispatching RESOLVED and dispatching an unknown ID were rejected.
No route during dispatch	Dedicated failure test confirmed that the complaint remained OPEN and no new event was published.

Recorded output excerpts

PASS: singleton identity
PASS: adapter units
PASS: shortest chooses A
PASS: fastest chooses B
PASS: empty route list
PASS: subscribers and citizen filtering
PASS: dispatch fastest and change status
PASS: resolve and unsubscription
PASS: no-route dispatch
PASS: no-route preserves OPEN
All checks completed.

These are excerpts; the full run also prints other passing checks and interleaved citizen/dashboard messages. U1 receives OPEN, IN_PROGRESS, and RESOLVED for C1. The dashboard receives only the first two because it is removed before resolution.

Design rationale and limitations

The factory isolates service creation, the strategies isolate route-selection policy, the adapter isolates provider data formatting, and observers separate status changes from notification delivery. The repository offers a shared storage access point within this program.

This simulation omits real dispatch, authentication, durable storage, live traffic, delivery retries, and concurrent updates. It does not prioritize real emergency incidents. Alternative solutions are acceptable when they satisfy the rules and preserve traceability across all five phases.